

National University of Singapore
School of Computing
CS1101S: Programming Methodology
Semester I, 2026/2027

R3
Higher-Order Functions and Scope

Review

1. A *conditional statement* has the form

```
if expression:
    statement
else:
    statement
```

or

```
if expression:
    statement
elif expression:
    ...
else:
    statement
```

2. *Lambda expressions* can be used to create functions. They have the form

```
lambda parameters: expression
```

3. Functions are values:

- Functions can be passed as arguments to functions.
- Functions can return functions as returned values.

4. *Higher-order functions* are functions that manipulate and generate functions. Higher-order functions can be used to abstract common programming patterns.

5. Scoping rules of Python §1:

- All names in Python must be declared
 - as primitive names (including functions),
 - in declaration assignments,
 - as parameters of function definitions and lambda expressions, or
 - as function name of function definitions.
- A name occurrence refers to the *closest surrounding declaration* of the name.

6. The following higher-order function `sum` was discussed in the lecture:

```
def sum(term, a, next, b):
    return (0
            if a > b
            else term(a) + sum(term, next(a), next, b))
```

Note: It is not necessary to memorize this declaration, or even the name of the function. Declarations of such functions (if they are used) will be given in an Appendix or in the questions for examinations. What is more important is to be able to *read* such declarations and understand how the functions work.

Problems:

1. Use the substitution model to “manually” run the following program.

```
z = 1

def f(g):
    z = 3
    return g(z)

f(lambda y: y + z)
```

2. Given the following Python program:

```
x = 5

def f(g):
    x = 3
    return lambda x: x + g(x)

def g(f, y):
    h = lambda y, f: y(f)
    return h(f, y)
```

- What names are declared by this program?
- Which declaration does each name occurrence refer to?
- What is the value of `(f(lambda x: 2 * x))(4)`?
- What is the value of `g(lambda y: y + 2, x)`?

3. Write a function `my_sum` that computes the following sum, for $n \geq 1$, without higher-order functions:

$$1 \times 2 + 2 \times 3 + \dots + n \times (n + 1)$$

4. Does the function `my_sum` as defined in Question 3 give rise to a recursive process or an iterative process? What is the order of growth in time and in space, using Θ notation?

5. We can also define `my_sum` in terms of the higher-order function `sum`. Complete the declaration of `my_sum` below. You cannot change the definition of `sum`; you may only call it with appropriate arguments.

```
def my_sum(n) :  
    return sum(<T1>, <T2>, <T3>, <T4>)
```

T1:

T2:

T3:

T4:

6. Write an iterative version of `sum`.

